Aprendizaje por Refuerzo

Unidad 4 · Cuaderno de trabajo resuelto: teoría, pasos y código

Aprendizaje Automático · Maestría en Inteligencia Artificial · TecNM Virtual
Alumno(a): Nombre Apellido

Sección	Contenido
1	Teoría general paso a paso y esquema
2	Actividad 25 · Elementos del RL en tres ejemplos
3	Actividad 26 · Entorno propio (AspiBot)
4	Actividad 27 · Una iteración de Q-learning
5	Actividad 28 · Agente y elección de algoritmo
6	Actividad 29 · Nueva tabla Q (S3: +3 → +6)
7	Actividad 30 · Agente propio (SkyBot)

Código ejecutable en Google Colab: github.com/alixpeg0/aprendizaje-por-refuerzo-unidad4. Las salidas mostradas se obtuvieron ejecutando el código de este documento.

1. Teoría general paso a paso

Paso 1. ¿Qué es el aprendizaje por refuerzo?. Un **agente** aprende a decidir **interactuando** con un entorno. No recibe ejemplos etiquetados: recibe **recompensas** y descubre por ensayo y error qué acciones convienen. Su meta es maximizar la recompensa **total acumulada**.

Paso 2. Ciclo agente–entorno. El agente observa el estado `s`, elige una acción `a`; el entorno responde con recompensa `r` y nuevo estado `s'`; el agente aprende de `(s, a, r, s')` y repite.

Paso 3. Seis elementos. Medio ambiente, **estado**, **acción**, **recompensa** (señal inmediata), **política** π(s) (regla de decisión) y **valor** V(s) / Q(s,a) (recompensa futura esperada).

Paso 4. Retorno y descuento γ. El agente maximiza $G = r_1 + \gamma \cdot r_2 + \gamma^2 \cdot r_3 + \dots$ con $0 \le \gamma \le 1$. Con $\gamma = 0.9$, una recompensa a 2 pasos vale 0.81.

Paso 5. MDP. Modelo formal (S, A, P, R, γ). Propiedad de Markov: el futuro depende **solo del estado actual**.

Paso 6. Ecuación de Bellman. Valor = recompensa inmediata + valor descontado de lo que sigue: $Q^*(s,a) = r + \gamma \cdot \max Q^*(s',a')$.

Paso 7. Q-learning. $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max Q(s',a') - Q(s,a) \,]$. El corchete es el **error TD**. Es *off-policy*.

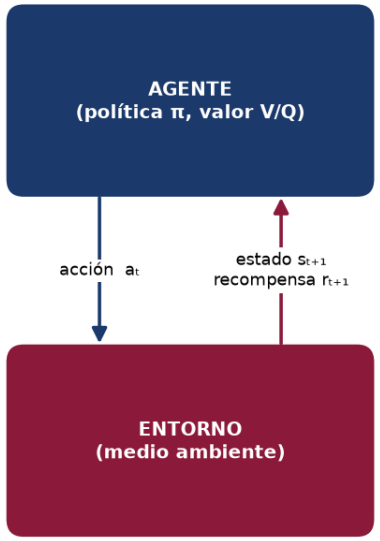
Paso 8. ε-greedy. Con prob. ε explora (acción al azar); con 1−ε explota (mayor Q). Se suele reducir ε con el tiempo.

Paso 9. Familias de algoritmos. Valores (Q-learning, DQN), políticas (REINFORCE, PPO), actor-crítico (A2C, SAC), modelos (Dyna-Q, MCTS).

Paso 10. Tipos de refuerzo. Positivo: se añade un premio. **Negativo:** se retira algo desagradable. **Castigo:** se añade algo desagradable.

Esquema de la teoría

Aprendizaje por refuerzo: esquema de la teoría



Ciclo: observar → actuar → recibir recompensa → actualizar

Elementos

Estado s	lo que observa el agente
Acción a	decisión que puede tomar
Recompensa r	premio (+) o castigo (−)
Política π(s)	qué acción elegir en cada s
Valor V(s)	retorno esperado desde s
Valor Q(s,a)	retorno esperado de (s, a)

Familias de algoritmos

- **Basados en valores** — Q-learning, SARSA, DQN
- **Basados en políticas** — REINFORCE, PPO
- **Actor-crítico** — A2C, SAC
- **Basados en modelos** — Dyna-Q, MCTS

Ideas matemáticas

Retorno: $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$

Bellman: $Q^*(s,a) = r + \gamma \max_{a'} Q^*(s',a')$

Q-learning: $Q(s,a) \leftarrow Q + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q\,]$

Tipos de refuerzo

- **Positivo** — añadir premio (+r)
- **Negativo** — quitar molestia (o penalizar −r)

Proceso de Markov (MDP): (S, A, P, R, γ) — el futuro solo depende del estado actual

2. Actividad 25 · Detecta los elementos del aprendizaje por refuerzo

Objetivo: identificar y comprender los conceptos básicos del RL en situaciones reales.

Teoría necesaria: cómo reconocer cada elemento

Elemento	Pregunta guía	Definición detallada	Error frecuente
Agente	¿Quién decide y aprende?	Entidad que observa, elige acciones y mejora con la experiencia.	Confundirlo con el usuario o con todo el sistema.
Medio ambiente	¿Qué responde a sus acciones y no controla directamente?	Todo lo externo al agente: reglas, física, otros actores. Produce el siguiente estado y la recompensa.	Meter dentro del agente cosas que no puede controlar.
Estado s	¿Qué información tiene para decidir?	Resumen de la situación. Idealmente cumple la propiedad de Markov (contiene todo lo relevante).	Incluir datos irrelevantes o dejar fuera datos clave.
Acción a	¿Qué puede hacer directamente?	Conjunto de decisiones disponibles (discretas o continuas).	Listar resultados en lugar de decisiones.
Recompensa r	¿Qué número le dice si le fue bien o mal?	Escalar que entrega el entorno tras cada acción. Define el objetivo del problema.	Confundirla con el valor o con el estado.
Política π(s)	¿Cómo elige su acción?	Función de estado a acción (determinista) o a probabilidades (estocástica). Es lo que se aprende.	Describir una acción suelta en vez de una regla.
Valor V(s), Q(s,a)	¿Qué tan bueno es estar aquí a largo plazo?	Retorno futuro esperado: V(s) = E[G | s]. No es una señal del entorno: lo estima el agente.	Tomar la recompensa inmediata como valor.

Recompensa vs. valor (la diferencia más importante)

La **recompensa** es lo que ocurre *ahora*; el **valor** es lo que se *espera acumular desde ahora*. En ajedrez, capturar una pieza da recompensa inmediata, pero si deja al rey expuesto, el estado resultante tiene **valor bajo**. Por eso el agente no es voraz: elige por valor, no por recompensa inmediata.

Tipos de recompensa y diseño

- **Dispersa:** solo aparece al final (ej.: +100 al estacionar). Es fiel al objetivo, pero el aprendizaje es lento.
- **Densa:** guía cada paso (ej.: +0.1 por acercarse al cajón). Acelera el aprendizaje, pero mal diseñada genera *reward hacking* (el agente explota el premio sin cumplir el objetivo real).
- **Hipótesis de la recompensa:** cualquier objetivo puede expresarse como maximizar la recompensa acumulada esperada.

Cómo se resuelve

Paso 1. Para cada ejemplo, pregunta **quién aprende** (agente) y **con qué interactúa** (medio ambiente).

Paso 2. Describe **qué observa** (estado) y **qué puede hacer** (acciones).

Paso 3. Define la **señal numérica** que indica si lo hizo bien o mal (recompensa), con valores concretos.

Paso 4. Redacta la política (cómo decide) y el valor (qué tan buena es una situación a futuro).

Paso 5. Explica el papel de la recompensa en cada caso.

Resultado

Elemento	a) Coche autónomo	b) Videojuego	c) Recomendador
Agente (quien aprende)	El sistema de conducción del coche	El personaje controlado por la IA	El algoritmo recomendador
Medio ambiente (entorno)	Estacionamiento: cajones, otros autos, bordillos, física del vehículo	El nivel del juego: plataformas, monedas, enemigos, reglas	La plataforma y sus usuarios (catálogo, gustos, hábitos)
Estado (s)	Posición, orientación y velocidad del auto; distancias medidas por sensores	Posición del personaje, de los enemigos y de las monedas; vidas restantes	Perfil del usuario, historial de películas vistas, hora y dispositivo
Acción (a)	Girar el volante, acelerar, frenar, avanzar o retroceder	Moverse izquierda/derecha, saltar, esperar	Recomendar la película X (o una lista de ellas)
Recompensa (r)	+100 al quedar bien estacionado; −50 por chocar; −1 por cada segundo que pasa	+1 por moneda; +50 por llegar a la meta; −10 por tocar un enemigo	+1 si el usuario la ve completa; +0.5 si da clic; −1 si la ignora o abandona
Política (π)	Regla que decide la maniobra según lo que miden los sensores en cada instante	Regla que decide cuándo saltar o avanzar para tomar monedas esquivando enemigos	Regla que elige qué película mostrar a cada perfil de usuario
Valor (V o Q)	Qué tan prometedora es una posición: alto si ya está alineado y cerca del cajón	Qué tan bueno es estar en cierto punto del nivel: alto cerca de monedas y lejos de enemigos	Satisfacción futura esperada del usuario si se le muestra esa película (no solo el clic inmediato)

Papel de la recompensa en cada caso

a) Coche autónomo. La recompensa le dice al coche qué es estacionarse bien: premia el éxito final y castiga choques y tardanza. Con ella el auto descubre por ensayo y error que las maniobras suaves y precisas son las que más puntos acumulan en total.

b) Videojuego. Las monedas y la meta empujan al personaje a explorar y avanzar, mientras que el castigo por tocar enemigos le enseña a evitarlos. La recompensa es la única guía: nadie le explica las reglas, las deduce de lo que gana o pierde.

c) Recomendador. La recompensa mide si la recomendación fue buena (el usuario la vio o la ignoró). Así el sistema ajusta sus sugerencias a cada persona y, si la recompensa incluye el largo plazo, evita recomendar solo lo que genera clics rápidos.

Para comprobar que la recompensa es una señal numérica, se programa la del ejemplo a):

Código

```
def recompensa_coche(estacionado, choco):
    r = -1                  # cada segundo cuesta un punto (hay que ser rápido)
    if choco:
        r -= 50             # castigo por chocar
    if estacionado:
        r += 100            # premio por terminar bien
    return r

for situacion, args in {"maniobrando": (False, False), "chocó": (False, True),
                        "estacionó bien": (True, False)}.items():
    print(f"{situacion:15s} -> r = {recompensa_coche(*args):4d}")
```

Salida al ejecutarlo

```
maniobrando     -> r =   -1
chocó           -> r =  -51
estacionó bien  -> r =   99
```

3. Actividad 26 · Diseña tu propio entorno de aprendizaje por refuerzo

Objetivo: aplicar los conceptos básicos del RL de forma creativa.

Teoría necesaria: cómo se diseña un problema de RL

Diseñar un entorno de RL es **formular un MDP**: decidir qué ve el agente, qué hace y qué se le premia. Un mal diseño produce un agente que aprende bien... algo equivocado.

Paso 1. Define el objetivo en lenguaje natural (llegar a la suciedad con poca batería).

Paso 2. Elige el estado: solo lo necesario para decidir. Si hay 25 celdas × 3 niveles de batería × 2 (sucio/limpio) → 150 estados: aún cabe en una tabla.

Paso 3. Elige las acciones: pocas, claras y bajo control directo del agente.

Paso 4. Diseña la recompensa: premia el objetivo (+10), cobra los costos (−1 por paso) y penaliza lo que quieres evitar (−3 por choque). Cuida las magnitudes: si el choque costara menos que el paso, no aprendería a evitarlos.

Paso 5. Define el fin del episodio: al limpiar o al superar un máximo de pasos.

Paso 6. Elige la política de exploración: ε-greedy con ε decreciente: $\varepsilon = \max(\varepsilon_min, \varepsilon_0 \cdot (1 - episodio / (0.7 \cdot N)))$.

Paso 7. Define la evidencia de aprendizaje antes de entrenar (métricas).

Cómo saber si el agente aprendió

- **Retorno por episodio** (suma de recompensas): debe subir y estabilizarse. Se suaviza con media móvil.
- **Pasos por episodio:** debe bajar hasta el mínimo posible.
- **Política greedy** (ε = 0): ejecutarla y comprobar que llega siempre por la ruta corta, sin choques.
- **Comparar con el óptimo teórico:** aquí la ruta mínima tiene 6 movimientos: 5 pasos (−1 cada uno) + el último que limpia (+10) = **+5**.

Descripción y diseño

AspiBot, el robot aspiradora que aprende su ruta. AspiBot es un robot aspiradora doméstico que no trae el mapa de la casa programado: lo va descubriendo mientras trabaja. Su propósito es llegar a las zonas sucias en el menor número de movimientos posible, cuidando su batería y evitando muebles. Al inicio se mueve casi al azar y choca con frecuencia; cada vez que limpia recibe un premio y cada movimiento o choque le cuesta puntos. Después de muchos intentos aprende la ruta más corta y segura desde su base hasta la suciedad, y la repite aunque cambien algunos detalles de la casa.

Elemento	Diseño
Agente	AspiBot, robot aspiradora.
Medio ambiente	Una casa dividida en una rejilla de 5×5 celdas, con muebles (obstáculos), una base de carga en la esquina y una zona sucia.
Posibles estados	Su celda actual (fila, columna), nivel de batería (alto/medio/bajo) y si la zona sucia ya fue limpiada.

Elemento	Diseño
Acciones	Mover arriba, abajo, izquierda o derecha; aspirar.
Recompensas / penalizaciones	+10 al limpiar la zona sucia; −1 por cada paso (gasta batería); −3 por chocar con un mueble o pared; −20 si se queda sin batería lejos de la base.
Política o estrategia	ε-greedy sobre Q(s,a): al inicio explora mucho (ε alto) y con el tiempo explota (ε bajo), eligiendo la acción con mayor Q en cada celda.
¿Cómo sabré que aprendió?	El retorno por episodio sube y se estabiliza; el número de pasos baja hasta la ruta mínima; ya no choca; y la política greedy (sin exploración) llega a la suciedad siempre por el mismo camino corto.

Simulación para comprobar el aprendizaje

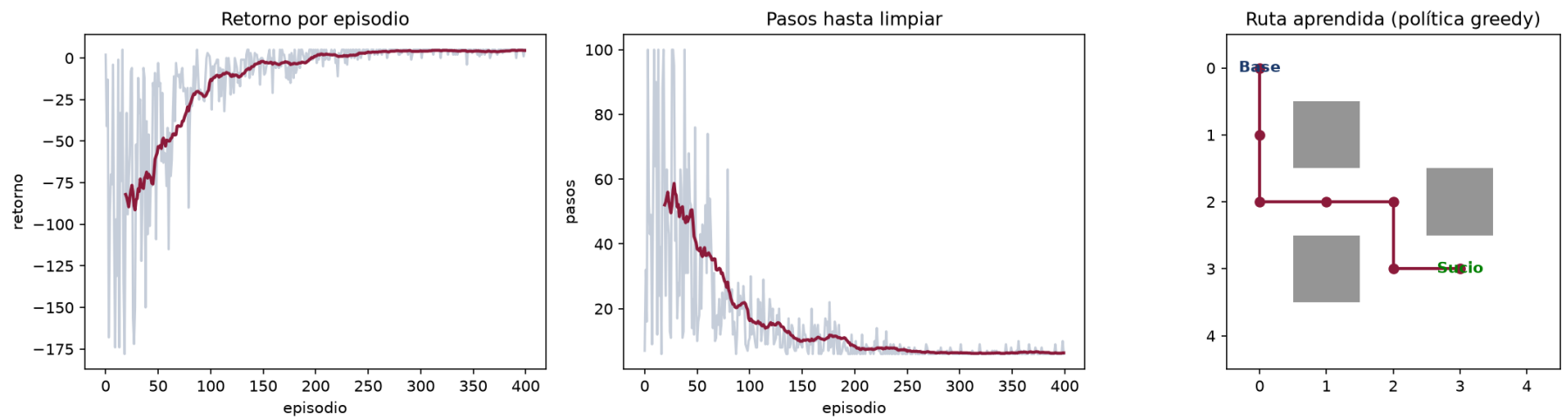
La rejilla está en `rl_core.py` (clase `Rejilla`). En cada paso el agente aplica la fórmula de Q-learning.

Código

```
env = Rejilla()                                  # 5x5, base (0,0), suciedad (3,3)
Q, retornos, pasos = entrenar_rejilla(env, episodios=400, alpha=0.5, gamma=0.95)
print("Retorno medio, primeros 20 episodios:", round(retornos[:20].mean(), 1))
print("Retorno medio, últimos 20 episodios :", round(retornos[-20:].mean(), 1))
print("Pasos medios, últimos 20 episodios  :", round(pasos[-20:].mean(), 1))
print("Ruta aprendida (greedy):", ruta_greedy(env, Q))
```

Salida al ejecutarlo

```
Retorno medio, primeros 20 episodios: -82.0
Retorno medio, últimos 20 episodios : 4.6
Pasos medios, últimos 20 episodios  : 6.4
Ruta aprendida (greedy): [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3)]
```



Conclusión

Sabré que AspiBot aprendió correctamente cuando, sin explorar (ε = 0), llegue a la zona sucia en el mínimo de pasos de forma consistente, sin choques, y su retorno promedio por episodio se mantenga estable en un valor alto. En la simulación del notebook el retorno promedio pasa de ≈ −82 (primeros 20 episodios) a ≈ +4.5 (últimos 20), muy cerca de +5, el máximo posible en esa rejilla (5 pasos de −1 y la limpieza de +10).

4. Actividad 27 · Simula una iteración de Q-learning

Objetivo: aplicar la ecuación de Bellman para calcular Q(s,a). **Parámetros:** α = 0.5, γ = 0.9, Q = 0 al inicio.

Teoría necesaria: de Bellman a Q-learning

Paso 1. La tabla Q. Guarda una estimación Q(s,a) por cada pareja estado-acción: «cuánta recompensa total espero si en s hago a y luego actúo bien». Se inicia en 0 (no sabe nada).

Paso 2. Idea de Bellman. El retorno se descompone: `G = r + γ·G'`. Por tanto, el valor de (s,a) es la recompensa inmediata más γ por el mejor valor posible en el estado siguiente: `Q(s,a) = r + γ·max Q(s',a')`.

Paso 3. Problema: al inicio Q es inexacta, así que esa igualdad no se cumple. La diferencia se llama **error TD**: `δ = [r + γ·max Q(s',a')] − Q(s,a)`.

Paso 4. Corrección parcial. Se mueve Q hacia el objetivo un paso α: `Q ← Q + α·δ`, equivalente a un promedio ponderado `Q ← (1−α)·Q + α·objetivo`. Con α = 0.5 se toma mitad de lo viejo y mitad de lo nuevo; α = 1 olvidaría todo lo anterior y α = 0 no aprendería.

Paso 5. Papel de γ. Con γ = 0.9 el futuro cuenta casi igual que el presente; el `max` supone que a partir de s' el agente actuará lo mejor posible (por eso Q-learning es *off-policy*).

Paso 6. Propagación hacia atrás. Un estado cercano a una recompensa gana valor primero; en las siguientes visitas, los estados que llevan a él heredan ese valor.

Paso 7. Convergencia. Si todas las parejas (s,a) se visitan muchas veces y α es adecuada, Q converge a Q*. Una iteración es solo el primer tramo.

Ejemplo anotado (paso 4 de la tabla)

Estado S3, acción Izquierda, `r = 5`, siguiente estado S2. Antes del paso, `Q(S3,Izq) = 0` y `max Q(S2,·) = 0.5`. Objetivo = 5 + 0.9·0.5 = **5.45**; error TD = 5.45 − 0 = 5.45; `Q = 0 + 0.5·5.45 =` **2.725**.

Cómo se resuelve

Paso 1. Define el entorno: 3 estados y 3 acciones con su recompensa y siguiente estado.

Paso 2. Inicia la tabla Q en 0.

Paso 3. Elige una trayectoria de 6 experiencias que visite los tres estados y repita algún par (s,a).

Paso 4. Para **cada** experiencia: objetivo = r + γ·max Q(s',·); error = objetivo − Q; Q nuevo = Q + α·error.

Paso 5. Registra cada cálculo en una tabla y localiza el valor Q más alto.

Paso 6. Interpreta el resultado.

Entorno

Estado s	Acción a	Siguiente s'	Recompensa r
S1	Izquierda	S1	-1
S1	Derecha	S2	0
S1	Quedarse	S1	0
S2	Izquierda	S1	0
S2	Derecha	S3	1
S2	Quedarse	S2	0
S3	Izquierda	S2	5
S3	Derecha	S3	-1
S3	Quedarse	S3	3

Código

```
T = construir_entorno(r_s3_quedarse=3)       # T[(s, a)] = (s', r)
Q = q_vacia()                                # tabla 3x3 con ceros

def actualizar(Q, s, a, r, s2, alpha=0.5, gamma=0.9):
    mejor_siguiente = Q.loc[s2].max()               # max Q(s', a')
    objetivo = r + gamma * mejor_siguiente          # lo que ahora creemos que vale
    error_td = objetivo - Q.loc[s, a]               # cuánto nos equivocamos
    Q.loc[s, a] += alpha * error_td                 # corrección parcial (alpha)
    return mejor_siguiente, objetivo, error_td

trayectoria = [("S1","Derecha"), ("S2","Derecha"), ("S3","Quedarse"),
               ("S3","Izquierda"), ("S2","Derecha"), ("S3","Izquierda")]
for i, (s, a) in enumerate(trayectoria, 1):
    s2, r = T[(s, a)]
    q_ant = Q.loc[s, a]
    m, obj, err = actualizar(Q, s, a, r, s2)
    print(f"Paso {i}: Q({s},{a}) = {q_ant:.4f} + 0.5*[{r} + 0.9*{m:.4f} - {q_ant:.4f}] = {Q.loc[s, a]:.4f}")

print()
print(Q.round(4))
print("\nQ más alto:", Q.stack().idxmax(), "=", round(Q.stack().max(), 4))
```

Salida al ejecutarlo

```
Paso 1: Q(S1,Derecha) = 0.0000 + 0.5*[0 + 0.9*0.0000 - 0.0000] = 0.0000
Paso 2: Q(S2,Derecha) = 0.0000 + 0.5*[1 + 0.9*0.0000 - 0.0000] = 0.5000
Paso 3: Q(S3,Quedarse) = 0.0000 + 0.5*[3 + 0.9*0.0000 - 0.0000] = 1.5000
Paso 4: Q(S3,Izquierda) = 0.0000 + 0.5*[5 + 0.9*0.5000 - 0.0000] = 2.7250
Paso 5: Q(S2,Derecha) = 0.5000 + 0.5*[1 + 0.9*2.7250 - 0.5000] = 1.9763
Paso 6: Q(S3,Izquierda) = 2.7250 + 0.5*[5 + 0.9*1.9763 - 2.7250] = 4.7518

    Izquierda  Derecha  Quedarse
S1     0.0000   0.0000       0.0
S2     0.0000   1.9762       0.0
S3     4.7518   0.0000       1.5

Q más alto: ('S3', 'Izquierda') = 4.7518
```

Tabla de cálculos

Paso	s	a	r	s'	Q ant.	max Q(s',·)	Objetivo	Error TD	Q nuevo
1	S1	Derecha	0	S2	0	0	0	0	0
2	S2	Derecha	1	S3	0	0	1	1	0.5
3	S3	Quedarse	3	S3	0	0	3	3	1.5
4	S3	Izquierda	5	S2	0	0.5	5.45	5.45	2.725
5	S2	Derecha	1	S3	0.5	2.725	3.4525	2.9525	1.9763
6	S3	Izquierda	5	S2	2.725	1.9763	6.7786	4.0536	4.7518

Conclusión

El valor Q más alto es **Q(S3, Izquierda) = 4.7518**. Significa que, con lo aprendido hasta ahora, la mejor decisión estando en S3 es ir a la izquierda: esa acción da +5 de inmediato y además conduce a S2, un estado desde el cual el agente ya descubrió que puede volver a S3 con recompensa (Q(S2,Derecha) = 1.9763). Es decir, un Q alto resume la **recompensa inmediata más el valor futuro descontado** de elegir esa acción.

Observa que el aprendizaje se **propaga hacia atrás**: en el paso 5, Q(S2,Derecha) saltó de 0.5 a 1.9763 porque el siguiente estado (S3) ya tenía un valor alto (2.725). Con una sola iteración los valores todavía son pequeños y están incompletos (Q(S1,·) sigue en 0); al repetir muchos episodios, Q converge a los valores óptimos de la Actividad 29 (por ejemplo Q*(S3,Izquierda) ≈ 31.05).

5. Actividad 28 · Diseña un agente y elige su algoritmo

Objetivo: aplicar los conceptos aprendidos para seleccionar un algoritmo adecuado.

Teoría necesaria: mapa de algoritmos de RL

Elegir algoritmo es contestar tres preguntas: **¿conozco o puedo simular el entorno?**, **¿cuántos estados tengo?** y **¿las acciones son discretas o continuas?**

Algoritmo	Cómo funciona	Fortaleza	Limitación
Q-learning (tabular)	Tabla Q(s,a) actualizada con la fórmula de Bellman. Off-policy.	Simple y con garantías de convergencia.	La tabla crece con el nº de estados: no escala.
SARSA	Como Q-learning, pero usa la acción a' realmente elegida (on-policy).	Más cauteloso con riesgos durante el aprendizaje.	Aprende la política que ejecuta, con exploración incluida.
DQN	Una red neuronal estima $Q(s,a;\theta)$. Pérdida: $(y - Q)^2$, con $y = r + \gamma \cdot \max Q(s',a';\theta^-)$. Usa **memoria de repetición** y **red objetivo**.	Maneja espacios de estados enormes o continuos.	Acciones discretas; entrenamiento sensible.
REINFORCE / PPO	Optimizan directamente la política: $\nabla J = E[\nabla \log \pi(a\|s) \cdot G]$. PPO limita cada actualización (clipping).	Acciones continuas, políticas estocásticas, estable (PPO).	Necesitan muchas muestras.
Actor-crítico (A2C, SAC)	El **actor** propone acciones; el **crítico** estima el valor y reduce la varianza.	Equilibrio entre estabilidad y eficiencia.	Más componentes que ajustar.
Dyna-Q	Tras cada paso real aprende un **modelo** del entorno y hace k pasos de planificación simulados.	Muy eficiente en datos cuando interactuar es caro.	Un modelo impreciso lleva a planes erróneos.

Criterios de decisión

- **Pocos estados y acciones discretas** → Q-learning tabular o SARSA.
- **Estados muy grandes o continuos, acciones discretas** → DQN.
- **Acciones continuas** (ángulos, fuerzas) → PPO, SAC.
- **Interactuar es caro pero se puede simular/modelar** → Dyna-Q u otros basados en modelos.

Cómo se resuelve

Paso 1. Describe el agente y el entorno (aquí: un semáforo inteligente en un cruce).

Paso 2. Lista las **acciones**: pocas y discretas (mantener / cambiar fase).

Paso 3. Diseña la **recompensa**: penalizar la espera total y los cambios de fase excesivos.

Paso 4. Estima el **tamaño del espacio de estados**: si es enorme, descarta Q-learning tabular.

Paso 5. Compara las familias con los criterios anteriores y elige.

Paso 6. Justifica la elección con las características del problema.

Apartado	Descripción
Agente	Controlador inteligente de un semáforo en un cruce de dos avenidas.
Descripción del entorno	Un cruce simulado (por ejemplo en SUMO) con flujo de autos variable según la hora. El estado incluye la longitud de la fila en cada carril, el tiempo de espera acumulado y la fase actual del semáforo; es un espacio muy grande (valores continuos).
Acciones posibles	Mantener la fase actual o cambiar a la siguiente fase (verde para el otro sentido). Son pocas acciones discretas.
Recompensas esperadas	Recompensa = −(tiempo total de espera de los autos) en cada intervalo; −5 por cambios de fase demasiado frecuentes; +10 cuando se vacía una fila larga.
Tipo de algoritmo elegido	Basado en valores con aproximación por red neuronal: DQN (Deep Q-Network).
Justificación	Con filas de longitud variable el número de estados es enorme, y una tabla Q como la de la Actividad 27 no cabría en memoria. DQN conserva la lógica de Q-learning pero estima Q(s,a) con una red neuronal. Como las acciones son pocas y discretas, un método basado en valores encaja bien (los basados en políticas, como PPO, se justifican sobre todo con acciones continuas). Además usa repetición de experiencias para aprender de forma estable.

Comparación aplicada a este agente

Algoritmo	Familia	Ideal cuando…	¿Para este agente?
Q-learning tabular	Valores	Pocos estados y acciones discretas	No: el espacio de estados es demasiado grande
DQN	Valores + red neuronal	Muchos estados, acciones discretas	Sí: es la opción elegida
PPO / REINFORCE	Políticas	Acciones continuas o políticas estocásticas	Posible, pero más complejo de lo necesario
Dyna-Q	Basado en modelo	Simulaciones baratas y estados discretos	Útil si se aprende un modelo del tráfico, pero el modelo es difícil de ajustar

Por qué no basta una tabla Q y cómo se calcula el objetivo de DQN

Código

```
import numpy as np
# --- 1) ¿Cabe una tabla Q? 4 carriles con filas de 0..30 autos, 4 fases, espera en 10 niveles
n_estados = (31 ** 4) * 4 * 10
print(f"Estados posibles: {n_estados:,}  x 2 acciones = {n_estados*2:,} celdas Q")

# --- 2) Recompensa del controlador
def recompensa_semaforo(espera_total, cambio_de_fase, fila_vaciada):
    return -espera_total - (5 if cambio_de_fase else 0) + (10 if fila_vaciada else 0)
print("r =", recompensa_semaforo(espera_total=12, cambio_de_fase=True, fila_vaciada=False))

# --- 3) Objetivo de DQN: y = r + gamma * max_a' Q_objetivo(s', a')  (con una red, aquí simulada)
gamma = 0.9
q_red_objetivo_siguiente = np.array([-14.2, -9.8])   # salida de la red para [mantener, cambiar]
r = recompensa_semaforo(12, True, False)
y = r + gamma * q_red_objetivo_siguiente.max()
print("Objetivo DQN y =", round(y, 2), " (la red se entrena para acercar Q(s,a) a y)")
```

Salida al ejecutarlo

```
Estados posibles: 36,940,840  x 2 acciones = 73,881,680 celdas Q
r = -17
Objetivo DQN y = -25.82  (la red se entrena para acercar Q(s,a) a y)
```

6. Actividad 29 · Calcula tu propia tabla Q

Objetivo: ver cómo cambiar una recompensa modifica los valores Q y la acción óptima.

Teoría necesaria: valores óptimos y sensibilidad a la recompensa

Paso 1. Política óptima. Dado Q*, la mejor política es codiciosa: $\pi^*(s) = \text{argmax}_a\, Q^*(s,a)$.

Paso 2. Cálculo de Q*. Se repite $Q(s,a) \leftarrow r + \gamma \cdot \max Q(s',a')$ para todas las parejas hasta que ya no cambie (*iteración de valor*). Converge porque cada barrido reduce el error por un factor γ.

Paso 3. Una iteración vs. Q*. Con una sola trayectoria la tabla es parcial; Q* es el valor al que tiende el aprendizaje completo.

Paso 4. Valor de quedarse siempre. Si se cobra r en cada paso: $V = r + \gamma \cdot r + \gamma^2 \cdot r + \ldots = r/(1-\gamma)$. Con γ = 0.9: V = 10·r (+3 → 30; +6 → 60).

Paso 5. Valor del ciclo S3→S2→S3. $V3 = 5 + 0.9 \cdot V2$ y $V2 = 1 + 0.9 \cdot V3$, luego $V3 = 5.9/(1 - 0.81) = 31.05$.

Paso 6. Umbral de cambio. Quedarse gana cuando $10 \cdot r \geq 5.9 + 8.1 \cdot r$, es decir $r \geq 5.9/1.9 \approx 3.105$. Con +3 el ciclo gana por poco; con +6 gana quedarse.

Paso 7. Lección de diseño. Cuando dos opciones están casi empatadas, un cambio pequeño en la recompensa invierte la política. Por eso la recompensa debe diseñarse con cuidado.

Cómo se resuelve

Paso 1. Parte del mismo entorno de la Actividad 27.

Paso 2. Cambia la recompensa de (S3, Quedarse) de **+3 a +6**.

Paso 3. Recalcula Q con la **misma trayectoria** y compara con la original.

Paso 4. Calcula los valores **óptimos Q*** para ver la política final.

Paso 5. Compara políticas y explica qué cambió y por qué.

Código

```
T3, T6 = construir_entorno(3), construir_entorno(6)      # original y modificado

# (1) Una iteración con la misma trayectoria
Q_orig, _ = simular_trayectoria(T3)
Q_new,  _ = simular_trayectoria(T6)
print("Tras una iteración (original +3):"); print(Q_orig.round(4))
print("\nTras una iteración (nueva +6):");   print(Q_new.round(4))

# (2) Valores óptimos Q*: repetir Q(s,a) = r + gamma * max Q(s',a') hasta converger
Qs3, Qs6 = q_optima(T3, gamma=0.9), q_optima(T6, gamma=0.9)
print("\nQ* original (+3):"); print(Qs3.round(2))
print("\nQ* nueva (+6):");   print(Qs6.round(2))
print("\nPolítica original:", politica(Qs3).to_dict())
print("Política nueva   :", politica(Qs6).to_dict())

# (3) Comprobación con Q-learning entrenado por interacción
Qe = entrenar_q_learning(T6, episodios=500)
print("\nPolítica aprendida por Q-learning:", politica(Qe).to_dict())
```

Salida al ejecutarlo

```
Tras una iteración (original +3):
    Izquierda  Derecha  Quedarse
S1     0.0000   0.0000       0.0
S2     0.0000   1.9762       0.0
S3     4.7518   0.0000       1.5

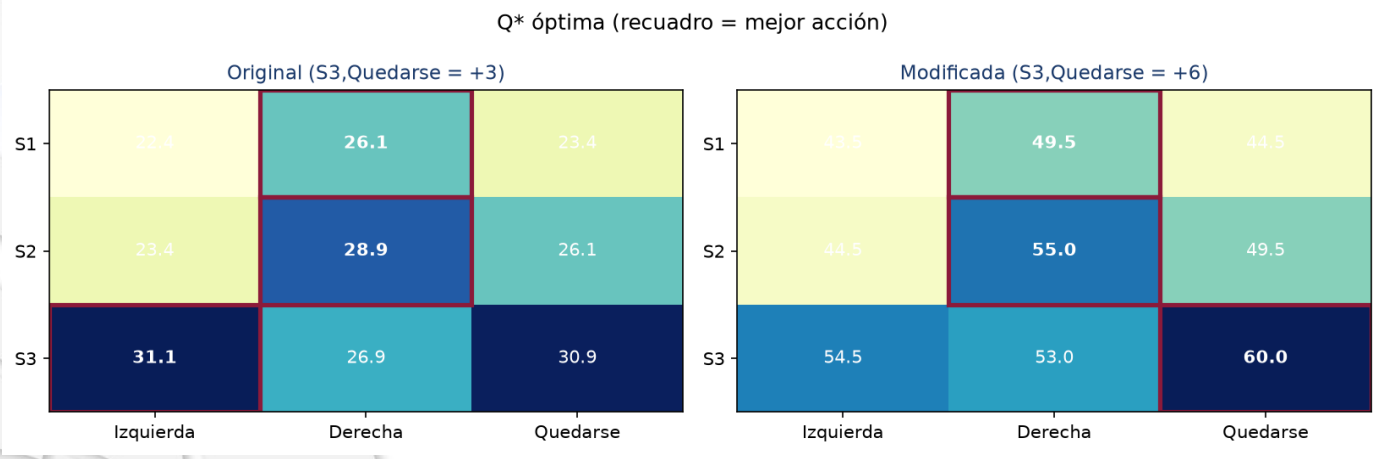
Tras una iteración (nueva +6):
    Izquierda  Derecha  Quedarse
S1     0.0000      0.0       0.0
S2     0.0000      2.1       0.0
S3     4.8075      0.0       3.0

Q* original (+3):
    Izquierda  Derecha  Quedarse
S1      22.45    26.05     23.45
S2      23.45    28.95     26.05
S3      31.05    26.95     30.95

Q* nueva (+6):
    Izquierda  Derecha  Quedarse
S1      43.55     49.5     44.55
S2      44.55     55.0     49.50
S3      54.50     53.0     60.00

Política original: {'S1': 'Derecha', 'S2': 'Derecha', 'S3': 'Izquierda'}
Política nueva   : {'S1': 'Derecha', 'S2': 'Derecha', 'S3': 'Quedarse'}

Política aprendida por Q-learning: {'S1': 'Derecha', 'S2': 'Derecha', 'S3': 'Quedarse'}
```



Análisis

¿Qué cambió en el comportamiento? En S3 la acción óptima pasó de **Izquierda** (Q* = 31.05 frente a 30.95 de Quedarse, casi empatadas) a **Quedarse** (Q* = 60 frente a 54.5). Antes al agente le convenía ir y venir entre S2 y S3 cobrando +5 y +1; ahora se queda en S3 cobrando +6 en cada paso. En S1 y S2 sigue yendo a la derecha, pero todos los valores Q suben porque llegar a S3 vale mucho más.

¿Por qué la nueva recompensa cambió la acción óptima? Porque el valor de una acción es la recompensa inmediata más el valor futuro descontado. Quedarse en S3 produce 6 + 0.9·V(S3), es decir V(S3) = 6/(1−0.9) = 60, mientras que el ciclo S3→S2→S3 rinde solo 5.9 por cada dos pasos (≈ 54.5). Al duplicar la recompensa de Quedarse, esa opción supera al ciclo. Tras una sola iteración la tabla todavía no lo refleja del todo, pero con el entrenamiento completo de Q-learning la política converge a la de la solución exacta.

7. Actividad 30 · Diseña tu propio agente y aplica la lógica del aprendizaje por refuerzo

Objetivo: comprender estado, acción, recompensa, política y valor mediante un entorno propio.

Teoría necesaria: tipos de refuerzo

El RL se inspira en el **condicionamiento operante**: una conducta cambia según sus consecuencias. Hay cuatro casos:

	Conducta AUMENTA	Conducta DISMINUYE
Se añade un estímulo	**Refuerzo positivo**: añadir un premio (+20 por entregar el paquete).	**Castigo positivo**: añadir algo desagradable (−10 por chocar).
Se retira un estímulo	**Refuerzo negativo**: quitar algo molesto (al llegar a la estación desaparece la alarma de batería baja).	**Castigo negativo**: quitar algo bueno (perder puntos o tiempo de vuelo).

- **Cuidado con el nombre:** «negativo» no significa castigo. El refuerzo negativo **aumenta** la conducta (la que elimina lo desagradable).
- **En RL** todo se codifica como un número r: $r > 0$ favorece la acción, $r < 0$ la desalienta. Conceptualmente se distingue qué tipo de consecuencia representa.
- **Adaptación con el tiempo:** (1) ε baja y el agente explota más; (2) cada experiencia actualiza Q; (3) las acciones con mayor Q se vuelven más frecuentes; (4) la política converge a rutas cortas y seguras.
- **Evidencia de aprendizaje:** retorno medio alto y estable, pocos pasos por entrega, cero choques/caídas y política greedy consistente.

Cómo se resuelve

Paso 1. Imagina un agente: un dron repartidor (**SkyBot**).

Paso 2. Escribe su propósito en un párrafo.

Paso 3. Completa la tabla: agente, entorno, estados, acciones, recompensas, política y evidencia de aprendizaje.

Paso 4. Indica qué **tipo de refuerzo** usa y por qué.

Paso 5. Explica cómo **se adapta** con el tiempo.

SkyBot: dron repartidor que aprende a entregar paquetes

Propósito. SkyBot es un dron repartidor que debe entregar paquetes en un barrio sin conocer de antemano la mejor ruta. Por ensayo y error aprende a llegar al punto de entrega rápido, esquivar edificios y cables, y regresar a cargar antes de que se agote la batería.

Elemento	Diseño
Agente	SkyBot, dron repartidor
Medio ambiente	Barrio con edificios, cables, viento, una estación de carga y puntos de entrega

Elemento	Diseño
Posibles estados	Posición (x, y, altura), nivel de batería, si lleva paquete, distancia al destino
Acciones	Subir, bajar, avanzar en 4 direcciones, soltar paquete, volver a la estación
Recompensas o penalizaciones	+20 por entregar; +5 por cargar con batería baja; −1 por cada paso; −10 por chocar; −30 si cae por batería
Política (estrategia)	ε-greedy con Q(s,a): explora al inicio y después elige la acción de mayor valor; volver a cargar si la batería < 20 %
Cómo sabrás si aprendió correctamente	Entrega en pocos pasos de forma consistente, sin choques ni caídas, con retorno promedio estable y alto

Tipo de refuerzo y adaptación

SkyBot usa ambos tipos de refuerzo. El refuerzo positivo es el premio que se añade al entregar el paquete (+20) y al cargar la batería a tiempo; el refuerzo negativo aparece cuando llegar a la estación elimina el estado aversivo de batería baja, y además se aplican penalizaciones (−10 por chocar, −1 por paso) que desalientan conductas. Con el tiempo reduce la exploración (ε baja), actualiza sus valores Q con cada experiencia y su política converge hacia rutas más cortas, seguras y con recargas oportunas.

Recompensas y política ε-greedy de SkyBot

Código

```
import random
def recompensa_dron(entrego, choco, cayo, cargo_con_bateria_baja):
    r = -1                                  # cada paso gasta batería
    r += 20 if entrego else 0               # refuerzo POSITIVO: premio añadido
    r += 5 if cargo_con_bateria_baja else 0  # refuerzo NEGATIVO: desaparece la batería baja
    r -= 10 if choco else 0                 # penalización (castigo)
    r -= 30 if cayo else 0                  # penalización fuerte
    return r

def elegir_accion(Q_estado, acciones, epsilon):
    if random.random() < epsilon:           # explorar
        return random.choice(acciones)
    return max(acciones, key=lambda a: Q_estado[a])   # explotar: mayor Q

print("Entrega ........:", recompensa_dron(True, False, False, False))
print("Choca ..........:", recompensa_dron(False, True, False, False))
print("Carga a tiempo .:", recompensa_dron(False, False, False, True))
print("Cae sin batería :", recompensa_dron(False, False, True, False))

random.seed(0)
Q_estado = {"subir": 1.0, "avanzar": 4.5, "volver_a_cargar": 2.0}
for eps in (1.0, 0.5, 0.05):
    elecciones = [elegir_accion(Q_estado, list(Q_estado), eps) for _ in range(1000)]
    print(f"epsilon={eps:4}: elige 'avanzar' (la mejor) el {elecciones.count('avanzar')/10:.1f}% de las veces")
```

Salida al ejecutarlo

```
Entrega ........: 19
Choca ..........: -11
Carga a tiempo .: 4
Cae sin batería : -31
epsilon= 1.0: elige 'avanzar' (la mejor) el 32.4% de las veces
epsilon= 0.5: elige 'avanzar' (la mejor) el 68.5% de las veces
epsilon=0.05: elige 'avanzar' (la mejor) el 96.4% de las veces
```

Al bajar ε, SkyBot explota cada vez más lo aprendido: así se refleja que su comportamiento se adapta con el tiempo.